ParcelPilot Operations and Incident Response Handbook

Triage, status checks, logs, configuration, incident notes, verification, and investigation practice.

ParcelPilot operations reference

Contents

1. Incident triage

Start with the user's exact symptom, approximate time, and scope. Write down what action failed and what still works before opening logs.

A broad statement such as 'ParcelPilot is broken' should be narrowed to login, order details, tracking, files, notifications, reports, or another concrete path.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

2. Incident state

Training incidents use open, investigating, and resolved. Open means recorded, investigating means checks are in progress, and resolved means the immediate issue has a supported explanation or corrective action that has been verified.

Do not mark resolved because one plausible cause was found. Verify the affected route after a change.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

3. Using service status

Health data is useful for scoping but is not a full end-to-end test. A service may be degraded while still handling some requests.

Use healthy, degraded, and down as operational labels. Compare the status capture time with the reported incident time.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

4. Reading logs

Filter logs by time and affected path. A generic API error may be a result of a more specific downstream error on the same request ID.

Do not select the most severe-looking error in the file. Select the one connected to the symptom.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

5. Configuration checks

Configuration tools should answer whether required settings are present and usable. For local paths, existence and readability matter.

Do not print secret values. A missing path is different from an unreadable file and from a database schema mismatch.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

6. Order incidents

For an order-page error, determine whether the basic order request fails or whether only tracking or documents fail. Test more than one known order when scope is unclear.

ORD-404, ORD-500, ORD-409, and ORD-503 point to different conditions and should not be treated as equivalent.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

7. Tracking incidents

Tracking incidents can be partial, stale, invalid, or slow. Inspect carrier snapshot status and tracking logs around the incident.

Keep available events visible when the state is partial, and do not invent a current event from older data.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Normal operating pattern

Normal tracking reads a recent carrier snapshot and returns a sequence of shipment events. Some orders legitimately have fewer events than others, so the number of events alone is not an error.

Failure pattern

TRK-206 means the response is incomplete, TRK-409 means the snapshot is too old, TRK-422 means the source cannot be used safely, and TRK-504 means the carrier lookup exceeded its timeout.

Worked example

A snapshot updated eight minutes ago with two events for one order may be valid partial data. A snapshot updated two hours ago should be treated as stale even if the JSON structure is valid.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

8. Auth incidents

One user's AUTH-423 should be treated differently from many users seeing AUTH-503. Account state and service availability answer different questions.

Avoid repeated login attempts on a locked test account while investigating.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Normal operating pattern

Normal authentication traffic includes successful logins, occasional AUTH-401 responses for bad credentials, token refreshes, and a small number of expected account-lock events during testing.

Failure pattern

A service incident is more likely when known-good accounts fail together, AUTH-503 repeats across many request IDs, and the auth health state is degraded or down. A single AUTH-423 is an account-state problem until evidence shows broader impact.

Worked example

If one user is locked at 09:14 but two other known-good users sign in at 09:15, do not classify the event as an authentication outage.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

9. Document and notification incidents

A missing file, an unreadable file, a queue backlog, and provider outage have different evidence. Keep the investigation on the path that reproduces the user symptom.

A healthy order lookup does not rule out document or notification problems.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Normal operating pattern

Document routes first confirm the order and then look for the requested file under the configured store. Some orders do not have proof-of-delivery or customs documents, so a controlled not-found response can be normal.

Failure pattern

DOC-404, DOC-403, and DOC-500 should remain separate. A missing file, an unreadable file, and an invalid document root have different causes and different next checks.

Worked example

If label.pdf opens but proof.pdf returns DOC-404, do not change the global document-store path. Verify whether proof.pdf exists for that order.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

10. Performance incidents

Use a time window and repeated durations. One slow request is not enough to establish a service-wide issue.

Compare the same endpoint over several requests and check known large-query or provider-timeout conditions.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

11. Incident notes

A useful incident note contains symptom, scope, checks, evidence, current conclusion, unknowns, and next step.

Reference useful log rows or request IDs instead of pasting thousands of lines.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.

12. Verification

After a corrective action, rerun the actual affected flow. A green service status is useful but does not replace route verification.

If only a workaround was applied, record that clearly and keep any permanent follow-up separate.

Working method

- Start with the smallest reproducible case.
- Change or check one thing at a time.
- Record what the evidence rules in and rules out.

Common mistake

A common troubleshooting mistake is to jump from a user-facing symptom directly to a remembered root cause. Similar symptoms can come from different services, so current status, logs, and configuration should be checked first.

Handoff note

Write the note so another person can continue the investigation without asking what was already checked. Keep conclusions separate from guesses.

Operational example

A useful incident sequence is: reproduce the symptom, identify the backend route, check the owning service, correlate logs by request ID, inspect the relevant configuration or snapshot, make one corrective change, then rerun the original case.

During the investigation, note negative evidence as well. If authentication succeeds repeatedly during an order failure, that fact helps keep the incident focused on the order path.

Review questions

- Does the current conclusion explain the reported scope?
- Is there a more specific downstream error than the API error?
- Did a recent change affect the same dependency?
- Has the original failing request been verified after the change?

The strongest incident notes make it obvious which facts are observed, which conclusion is supported, and which follow-up remains open.